Jackpots

This document explains the configuration and Game Server integration for Transaction and Game Jackpots. The two variants differ in who controls the outcome:

Transaction Jackpot: outcome (including any win) is resolved by the platform prior to gameplay, then communicated to the game.

Game Jackpot: the game fully controls jackpot contributions and wins during the round.

Each campaign can set up multiple independent pools, with independent parameters and win conditions.

Transaction Jackpot

Transaction Jackpot is configured to determine outcomes based on Withdraw transaction amount. Then the outcome of wins randomisation is passed to the game server to align it with the game math. If multiple pools are configured, each win is evaluated independently, so game needs to be able to support e.g. Minor and Major jackpots won at the same round.

Parameters

Each pool has the following configuration:

1. `contributionRate` - defines what fraction of the `bet` should be added as `contribution` (accrual) to the Jackpot pool.
2. `seedContributionRate` - defines what fraction of the `bet` should be added as `seedContribution` to the Jackpot seed pool (a pool that is added to the pool right after it was won, along with the `reset` value).
3. `probability` - probability of triggering the Jackpot Win for a `bet = 1` (base currency). Actual trigger probability scales linearly with `bet` amount i.e. `triggerProbability = bet * probability`.
4. `reset` - fixed value paid additionally to each triggered jackpot.

Campaign also specify `baseCurrency` that pool amounts will be stored in. Setting campaign segmentation to a specific currency along with specifying the same `baseCurrency` will effectively disable multi-currency support and prevent any conversions affected by current exchange rates fluctuations.

Game API

If any of the Jackpot pools was won, the result is passed to the game engine to forward it for presentation in accordance with specific game rules. The input for the game is passed in `promo` body param of the **Path**: `/api/games/{game}/play` endpoint. In GDK the `promo` is an argument of the `play` method respectively:

```
export interface IPromo {
    transactionJackpot?: {
        jackpotWin: number;
        baseCurrencyJackpotWin: number;
        poolWins?: {
            [poolName: string]: {
                amount: boolean;
                baseCurrencyAmount: number;
            };
        };
    };
}
```

1. `jackpotWin` - amount paid to the player in player's currency (to be presented to the player), conversions rates are updated along with the platform Exchange Rates system,
2. `baseCurrencyJackpotWin` - informative amount in base currency,

3. `poolWins` - specify exact pools results.

The example game forwarding Transaction Jackpot result to the game client in data:

```
    play({bet, promo}, random) {
        const {win, gameplay} = evaluateRegularGameplay(bet, random);
        return {
            win, // transaction jackpot win is appended to the game win (don't add from promo)
            data: {
                ...gameplay,
                transactionJackpot: promo?.transactionJackpot,
            },
        };
    },
```

RTP

1. RTP added to the game per pool: `jackpotPoolRtp = contributionRate + seedContributionRate + probability * reset`.
2. Probability for a player to trigger a given pool scales linearly with the base currency bet level of `1`.

Example 100% RTP pool config:

```
{
    major: {
        contributionRate: 0.6
        seedContributionRate: 0.3
        reset: 1000
        probability: 0.0001
    }
}
```

Example additional 10% RTP pool added to game rtp:

```
{
    major: {
        contributionRate: 0.08
        seedContributionRate: 0.01
        reset: 10000
        probability: 0.000001
    }
}
```

Simulator

Although the RTP contribution added to the game can be calculated with a formulas above, game engine should be simulated with the Jackpot inputs, mainly to verify if engine doesn't throw errors when consuming promo data. Game can be simulated with Transaction Jackpot via the following command:

```
sudo npx slotify-gdk stats -g games/test-transaction-jackpot-game -c 10 -i 100m --tj '{"major":{"contributionRat
```

The `--tj '{"major":{"contributionRate":0.6,"seedContributionRate":0.3,"reset":1000,"probability":0.0001}}'` represent JSON configuration for the promotional campaign for each pool.

Statistics for added RTP and Hit Frequency for specific pools can be added to track sum of all pools, or targeting specific pools separately:

```
    transactionJackpotRtp: new TransactionJackpotRTP(),
    majorJackpotPoolRtp: new TransactionJackpotRTP("major"),
    transactionJackpotHf: new TransactionJackpotHF(),
    majorJackpotPoolHf: new TransactionJackpotHF("major"),
```

Feed

Calling `/feed/campaign/:campaignId?currency=${currency}` endpoint provides current pool amounts (intended for periodic updates in game clients and casino web-pages). The pool amounts will be converted to the specified `currency`:

```
{
    config,
    poolAmounts: {
        minor: 3120.23,
        major: 17203.5,
    }
};
```

Game Jackpot

Game Jackpot is configured only with the `reset` values for each pool (to be able to present `feed` with initial reset values even without game engine activity). Determining `contributions` and `seedContributions` is then handled fully by the game engine. This allows to implement more sophisticated Jackpot mechanics, without strict "linear" calculus required by the Transaction Jackpots.

Parameters

Each pool has the following configuration:

3. `reset` - fixed value paid additionally to each triggered jackpot.

Game API

Game doesn't receive any parameter on the `promo` input. If game is supposed provide any contributions or jackpot wins, then game engine is supposed to return the following `campaigns` data structure on the `play` return output:

```
export interface ICampaigns {
    gameJackpot?: {
        pools?: {
            [poolName: string]: {
                contribution?: number;
                seedContribution?: number;
                isJackpotWin?: boolean;
            };
        };
    };
}
```

1. `contribution` - amount to add to the given jackpot pool,
2. `seedContribution` - amount to add to the given jackpot seed pool,
3. `isJackpotWin` - specify if jackpot should be appended to the given wager win.

The example Game Jackpot result (emulating the mechanics of a Transactions Jackpot, but within game engine):

```
    play({bet}, random, betLimits) {
    const poolsConfig = {
        "minor": {contributionRate: 0.1, seedContributionRate: 0, probability: 0.5},
        "major": {contributionRate: 0.2, seedContributionRate: 0.1, probability: 0.1},
        "grand": {contributionRate: 0.3, seedContributionRate: 0.1, probability: 0.01},
    };

    const pools = Object.fromEntries(
        Object.entries(poolsConfig).map(([poolName, {contributionRate, seedContributionRate, probability}]) => [
            poolName,
            {
                contribution: contributionRate * bet,
                seedContribution: seedContributionRate * bet,
                isJackpotWin: random() / 2 ** 32 <= (probability * bet) / betLimits.exchangeRate,
            },
        ]),
    );

    return {
        win: 0,
        data: {
            pools,
            betLimits,
        },
        campaigns: {
            gameJackpot: {
                pools,
            },
        },
    };
```

Client API

The game engine doesn't know the exact Jackpot pool amount that will be released to the player. However, this value is appended after `play` execution to the game client response so it can be used for specific win presentation by the game front-end. If game server requested a Win for each pool, the following data structure will be appended to `play` response:

```
response.wager.promo.gameJackpot = {
    jackpotWin: number,
    baseCurrencyJackpotWin: number,
    poolWins: {[poolName]: {amount: number, baseCurrencyAmount: number}},
};
```

RTP

Entirely controlled by the game logics - requires mathematical analysis for a given model.

Feed

Same as in Transaction Jackpot.